Procesos de Desarrollo de Software

Pruebas de software

Sesión Práctica

Contenidos

1. Motivación.
2. JUnit.
3. Pruebas de caja negra.
4. Pruebas de caja blanca - JaCoCo.
5. Pruebas de integración – Mokito.
6. Pruebas de integración – Springboot.

Motivación

Repasar conceptos y definir estrategias para resolver el caso práctico

1. Motivación – ¿Qué es el testing?

Testing es el proceso de ejecutar un sistema con la intención de encontrar fallos.

Proceso

- Ejecutamos el software.
- Observamos su comportamiento.
- Lo comparamos con lo esperado.

Un **test exitoso** es aquel que hace fallar el programa.

2. Testing – Niveles

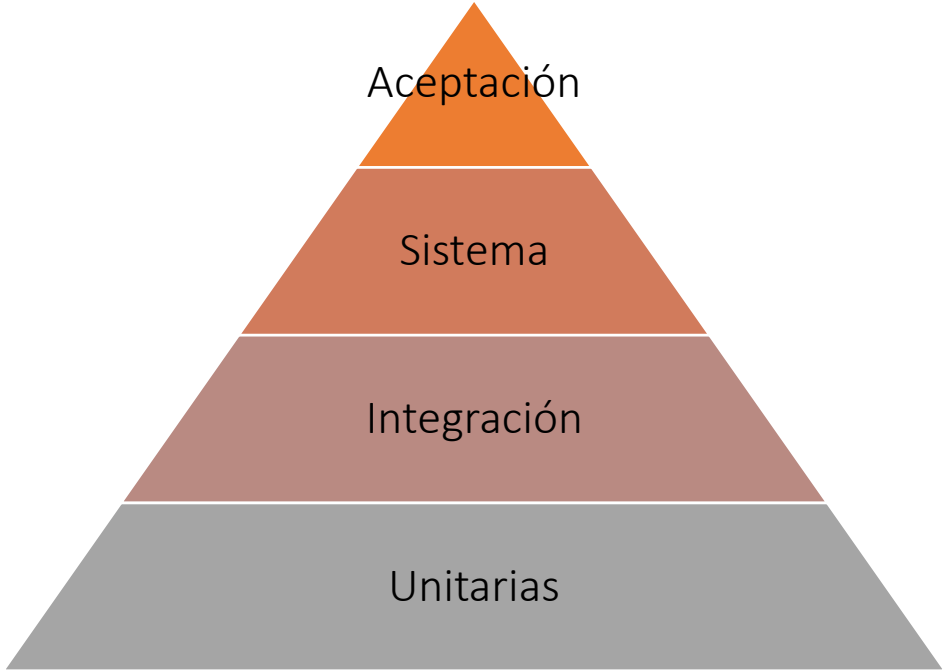
Pruebas Unitarias
Comprueban la funcionalidad de un módulo.

Pruebas de Integración
Comprueban los módulos al integrarse.

Pruebas de Sistema
comprueban requisitos no funcionales.

Pruebas de Aceptación
Comprueban requisitos del cliente.

Pruebas de regresión
Ejecutan todos los tests cuando se modifica el sistema.



2. Testing – Niveles

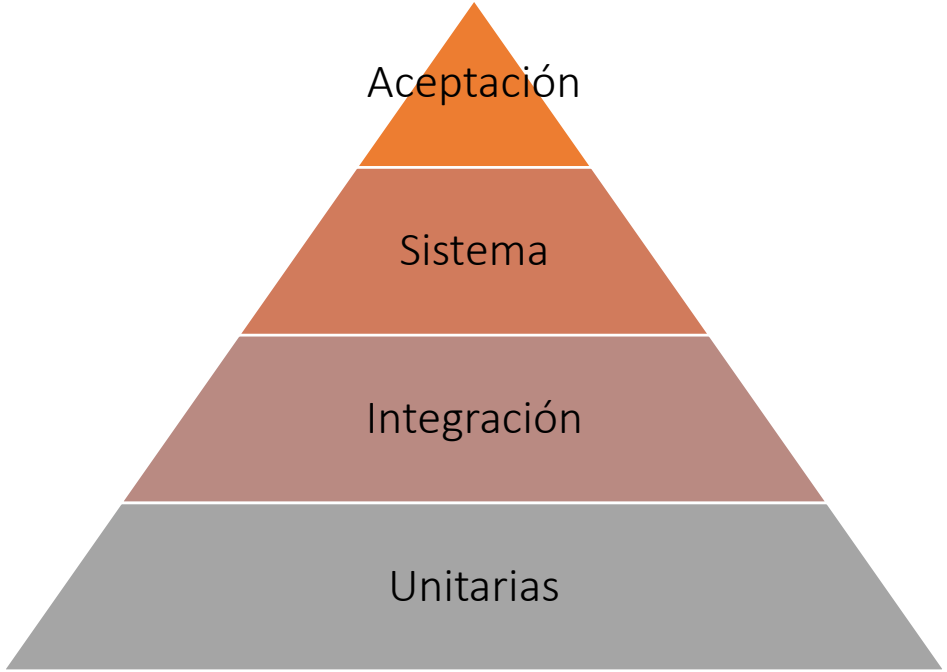
Pruebas Unitarias
Comprueban la funcionalidad de un módulo.

Pruebas de Integración
Comprueban los módulos al integrarse.

Pruebas de Sistema
comprueban requisitos no funcionales.

Pruebas de Aceptación
Comprueban requisitos del cliente.

Pruebas de regresión
Ejecutan todos los tests cuando se modifica el sistema.



2. Testing – Herramientas

JUnit

Incluir dependencias en POM

(si no lo están ya)

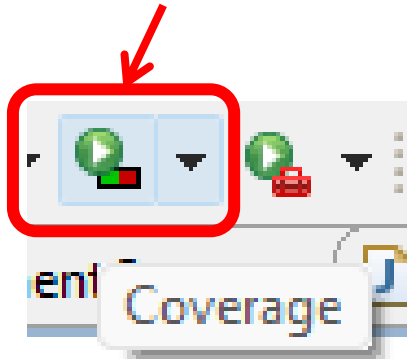
```
<properties>
  <junit.version>5.11.0</junit.version>
</properties>

<dependencies>
    <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>

  <!-- Soporte para tests parametrizados -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

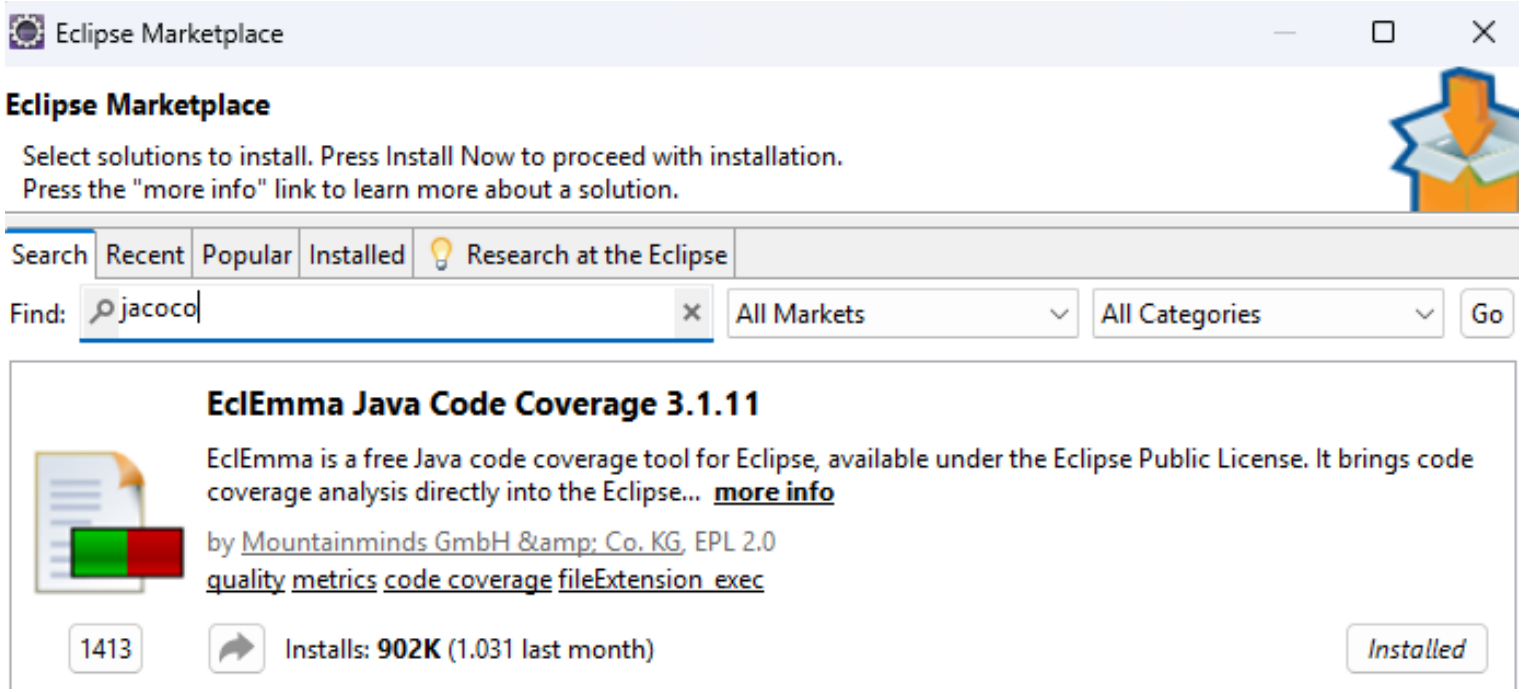
2. Testing – Herramientas

JaCoCo - EclEmma

Plugin para Eclipse. Instalarlo del Marketplace (si no está instalado ya)

JaCoCo instalado





JUnit

Testing framework

1. JUnit

JUnit 5

- Framework para realizar testing de manera automatizada.
- Definición de asertos → Se deben de cumplir para que el test termine correctamente.

Comprobación de igualdad	assertEquals(2,1+1)	assertNotEquals(3,1+1)
Comprobación de condición	assertTrue(5>3)	assertFalse(5<3)
Comprobación de referencia	assertSame(s1,s2)	assertNotSame(new String(“test”), new String ”test”)

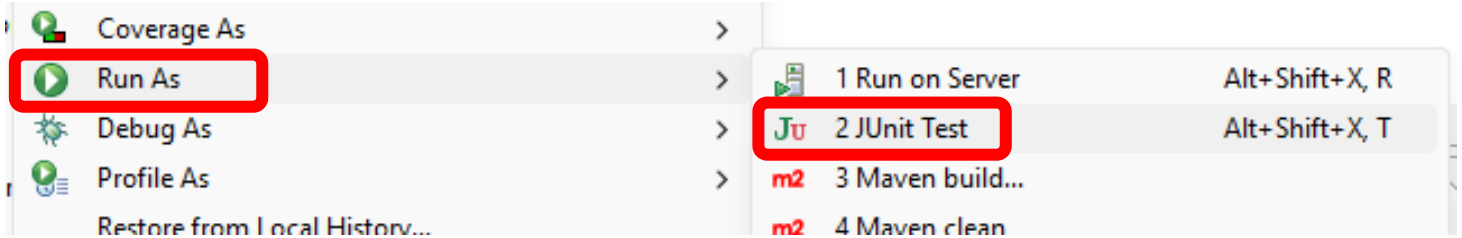
Excepciones	assertThrows(ArithmeticException.class, () -> {
	int result = 10 / 0;
	});	

Agrupaciones	assertAll("Persona", () -> assertEquals("John", "John"), () -> assertEquals("Doe", "Doe")); }	

1. JUnit

EJERCICIO

1. Crea el paquete pds.prueba en la carpeta src/test/java y copia el código del boletín.
2. Prueba a ejecutar la clase de prueba EjemploTest y examina los diferentes tipos de asertos.



Pruebas de caja negra

Usando JUnit

2. Pruebas de caja negra

Las **pruebas de caja negra** se centran en el comportamiento del sistema, sin tener en cuenta su implementación interna.

Características

- Se toma como base la especificación.
- Centradas en el dominio del problema, no en la implementación.
  - Capturan errores lógicos.
  - Se centra en los *comportamientos importantes* del software (los que están especificados)
- No es necesario conocer el código o que esté implementado.
- Se pueden aplicar a cualquier nivel de pruebas.

2. Pruebas de caja negra – Partition testing

Un conjunto de valores de entrada son una clase de equivalencia **(equivalence partition)** si se puede asumir de manera razonable que provocarán salidas *"similares"* al ejecutar la implementación.

Objetivo

- Definir grupos de entradas que provocan el mismo comportamiento en el programa.
  1. Leer la especificación
  2. Definir particiones
- Generación de casos de prueba concretos
  Elegir un valor de cada partición. Todos son equivalentes.

```
int abs(int valor) { … }
```

Clases de equivalencia

1. **valor < 0:** abs(-37) = 37, abs(-36) = 36, etc.
2. **valor > 0:** abs(10) = 10, abs(11) = 11, etc.
3. **valor = 0:** No es ninguna de las anteriores. abs(0) = 0

2. Pruebas de caja negra – Clases válidas e inválidas

Las entradas se pueden categorizar en:

- Clase válida: Entradas que el sistema debería aceptar y procesar correctamente.
- Clase inválida: Entradas que no cumplen los requisitos y el sistema debería rechazar con un error

Si la especificación incluye condiciones sobre la entrada *(ej., el valor tiene que ser mayor que 0)*, usar esta información para derivar las clases válidas e inválidas.

- Clase válida: dentro del rango.
- Clase inválida: fuera del rango (muy pequeño).
- Clase válida: fuera del rango (demasiado grande).

2. Pruebas de caja negra

Especificaciones:

- `ProductoId.of(..)`: Los códigos de producto <= 100 están reservados
- `Producto.tieneDescuento()`: Tiene descuento si stock < 3
- `Producto.getDescuento()`:
  - Si queda 1 producto -> 20% desc.
  - Si quedan 2 productos -> 10% desc.

EJERCICIO

Identifica las clases válidas e inválidas para la creación de value objects `ProductoId` y las **clases de equivalencia** para los métodos `Producto.tieneDescuento` y `Producto.getDescuento()`.

2. Pruebas de caja negra

Especificaciones:

- `ProductoId.of(..)`: Los códigos de producto <= 100 están reservados
- `Producto.tieneDescuento()`: Tiene descuento si stock < 3
- `Producto.getDescuento()`:
  - Si queda 1 producto -> 20% desc.
  - Si quedan 2 productos -> 10% desc.

EJERCICIO

Crea los casos de prueba. Crea clase ProductoTest (en src/test/java). La convención es poner la clase en el mismo paquete (inf.pds.tpv.domain.model.producto).

Para crear un test para una clase:

Botón derecho sobre el fichero de la clase, New -> JUnit Test Case

Pruebas de caja blanca

Usando JaCoCo

3. Pruebas de caja blanca – Definición

Las **pruebas de caja blanca** (o ***testing estructural***) son aquellas que se derivan del conocimiento el código fuente.

Motivación:

- Para encontrar un defecto, el código que lo contiene debe ejecutarse.
- Probar **funcionalidad no basta**.
  Debemos recorrer la estructura interna del código.
- Las pruebas de caja blanca nos ayudan a diseñar casos que ejerciten caminos y decisiones del código.

3. Pruebas de caja blanca – Herramientas

Utilizaremos JaCoCo (Java Code Coverage)

Ejecución con cobertura

Tb con BtnDcho
→Coverage As

→JUnit Test

Código cubierto por las pruebas ejecutadas

Métricas de cobertura

Poner a 0 las métricas

workspace - todo-list/src/main/java/pds/modelo/ToDoList.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer | JUnit

Finished after 0,33 seconds

Runs: 2/2 Errors: 0 Failures: 0

ToDoListTest [Runner: JUnit 5] (0,204 s)
- testGetPendientes() (0,171 s)
- testOrdenacionPorPrioridad() (0,032 s)

Failure Trace

ToDoItem.java | ToDoList.java | ToDoListTest.java

```
31    /**
32     * Dada una fecha, posiblemente la fecha actual, devuelve el número
33     * de tareas que están pendientes. No incluye aquellas tareas
34     * cuya fecha de vencimiento (si tienen) es menor que la fecha dada.
35     * De esa manera, las tareas que ya se han caducado no se consideran.
36     *
37     * @param desdeFecha La fecha a partir de la cual se consideran tareas.
38     * @return La lista de tareas pendientes
39     */
40    public List<ToDoItem> getPendientes(LocalDate desdeFecha) {
41        List<ToDoItem> pendientes = new ArrayList<ToDoItem>();
42        for (ToDoItem item: items) {
43            if (item.isTerminada()) {
44                continue;
45            }
46
47            if (item.getVencimiento() == null || item.getVencimiento().isAf
48                pendientes.add(item);
49            }
50        }
51        return pendientes;
52    }
```

Problems | Javadoc | Declaration | Console | Coverage

ToDoListTest (2 mar 2025 17:57:13)

Element	Coverage	Covered Instructior	Missed Instructi	Total Instructions
pds.modelo	72,7 %	160	60	220
ToDoItem.java	69,6 %	55	24	79
ToDoList.java	74,7 %	71	24	95
Etiqueta.java	0,0 %	0	12	12

3. Pruebas de caja blanca

EJERCICIO

Intentar conseguir un 100% de cobertura en los test ProductoTest y ProductoIdTest

Pruebas de integración

Fake object/stub, Mockito y Springboot

4. Pruebas de integración – Definición

Objetivo

Probar la interacción entre los módulos.

- Interés en problemas relativos a la coordinación entre módulos.
- Estas pruebas se realizan después las pruebas unitarias.

Las pruebas de integración en DDD:

- Implican probar las interacciones entre agregados, repositorios y otros servicios del dominio.
- Las pruebas de integración garantizan que sus agregados y servicios colaboren como se espera.

4. Pruebas de integración – Definición

Fake object

- Objetos que tienen una implementación “simulada”
- Usan una implementación más simple

Stub

- Similares a los Fake Objects pero en su lugar devuelven valores fijos

```
public class DatabaseFO implements Database {
  private List<Cliente> clientes;
  @Override
  public List<Cliente> getClientes() {
     return clientes
  }
}
```

Fake object

```
public class DatabaseStub implements Database {
  @Override
  public List<Cliente> getClientes() {
     return List.of(
         new Cliente(“c1@um.es”),
         new Cliente(“c2@um.es”));
  }
}
```

Stub

4. Pruebas de integración – Fake Object

Ejemplo de fake object / stub en el TPV (Probar en Eclipse):

```
@BeforeEach
void setUp() {
    repository = new ProductosRepositoryEnMemoria();
    servicio = new StockServiceImpl(repository);
}

@Test
void testCrearProducto() {
    CrearProductoCommand c = new CrearProductoCommand("prueba", 10, 100.0);
    Producto p = servicio.crearNuevoProducto(c);
    assertEquals("prueba", p.getDescripcion());
    …
}
```

```
private static class ProductosRepositoryEnMemoria implements ProductosRepository {
  private Map<ProductoId, Producto> datos = new HashMap<>();

@Override
public List<Producto> obtenerTodosProductos() {…}
 …
}
```

4. Pruebas de integración – Definición

Mock

- Similares a los Stub, pero además guardan las interacciones que se han realizado sobre ellos y se pueden consultar.

```
public class DatabaseMock implements Database {
  private numLlamadasGetClientes = 0;

  @Override
  public List<Cliente> getClientes() {
     numLlamadasGetClientes++;
     return List.of(
         new Cliente("c1@um.es"),
         new Cliente("c2@um.es"));
  }

  int getNumLlamadasGetClientes() { return numLlamadasGetClientes; }
}
```

4. Pruebas de integración – Mockito

Es un framework de mocking para Java

Simular el comportamiento de dependencias sin necesidad de crear implementaciones reales.

- Creación de mocks:
  - Se puede hacer manualmente con `Mockito.mock(Clase.class)`.
  - También se pueden usar anotaciones como `@Mock`.
- Definir comportamientos:
  - Se usa `when(mock.metodo()).thenReturn(valor)` para establecer respuestas predefinidas.
  - También se puede lanzar excepciones con `thenThrow()`.
- Verificación de interacciones:
  - Con `verify(mock).metodo()` se puede comprobar si un método fue llamado.
  - Se pueden verificar cuántas veces se llamó (`times(n)`) o si nunca se usó (`never()`).

4. Pruebas de integración – Mockito

EJERCICIO

- Probar el uso de Mockito para StockServiceImpl.crearProducto()
- Añadir pruebas para:
  - StockServiceImpl.eliminaProducto()
  - StockServiceImpl.editarProducto()

4. Pruebas de integración – SpringBoot

Tests de integración en SpringBoot:

- Permite realizar tests sin usar el resto de componentes de la aplicación (servidor, base de datos, etc.).
- Provee tanto anotaciones como mocks específicos.

Arranca SpringBoot en modo pruebas

```
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class StockEndpointTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;
```

4. Pruebas de integración – SpringBoot

Tests de integración en SpringBoot:

- Permite realizar tests sin usar el resto de componentes de la aplicación (servidor, base de datos, etc.).
- Provee tanto anotaciones como mocks específicos.

```
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class StockEndpointTest {
```

Prepara un objeto de tipo `MockMvc` para poder similar llamadas HTTP a la API

```
@Autowired
private MockMvc mockMvc;

@Autowired
private ObjectMapper objectMapper;
```

4. Pruebas de integración – SpringBoot

Tests de integración en SpringBoot:

- Permite realizar tests sin usar el resto de componentes de la aplicación (servidor, base de datos, etc.).
- Provee tanto anotaciones como mocks específicos.

Cada caso de prueba se ejecutará en una transacción diferente (se revertirá todo al terminar)

```
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class StockEndpointTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;
```

4. Pruebas de integración – SpringBoot

Tests de integración en SpringBoot:

- Permite realizar tests sin usar el resto de componentes de la aplicación (servidor, base de datos, etc.).
- Provee tanto anotaciones como mocks específicos.

```
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class StockEndpointTest {
```

Al ser un test, indicamos a SpringBoot que debe inyectar objetos(beans) aquí

```
@Autowired
private MockMvc mockMvc;
```

```
@Autowired
private ObjectMapper objectMapper;
```

4. Pruebas de integración – SpringBoot

Tests de integración en SpringBoot:

- Con MockMvc podemos simular una llamada a la API y testear su resultado

Simular la llamada

```
MvcResult result = mockMvc.perform(post(BASE + "/producto")
  .contentType(MediaType.APPLICATION_JSON)
  .content(json))
  .andExpect(status().isOk())
  .andReturn();
```

Deserializar la respuesta

```
String responseJson = result.getResponse().getContentAsString();
ProductoDTO producto = objectMapper.readValue(responseJson, ProductoDTO.class);

assertNotNull(producto);
assertTrue(producto.getCodigo() >= 131);
assertEquals("Papaya Nueva", producto.getDescripcion());
assertEquals(15, producto.getCantidad());
assertEquals(3.00, producto.getPrecio());
```

4. Pruebas de integración – SpringBoot

Tests de integración en SpringBoot:

- Con MockMvc podemos simular una llamada a la API y testear su resultado

Chequear sin deserializar

```
mockMvc.perform(post(BASE + "/producto")
            .contentType(MediaType.APPLICATION_JSON)
            .content(json))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.codigo").value(greaterThanOrEqualTo(131)))
    .andExpect(jsonPath("$.descripcion").value("Papaya Nueva"))
    .andExpect(jsonPath("$.cantidad").value(15))
    .andExpect(jsonPath("$.precio").value(3.00));
```

4. Pruebas de integración – SpringBoot

EJERCICIO

- Probar el test para StockEndpoint.createProducto() y arreglar el fallo
- Añadir casos de prueba para métodos GET, PUT, DELETE